From Linear Scores to a Single Hidden Layer

A Mathematical Study of Simple Learning Systems

Math4AI Final Capstone — National AI Center, AI Academy

Nazrin Aliyeva Nigar Rustamova

Laman Mirzayeva Saida Arabova

Team **AIngels** *(Artificial Intelligence Next-Gen Engineering Lean Squad)*

April 1, 2026

Abstract

This report investigates when a one-hidden-layer nonlinear classifier genuinely improves on a linear decision rule, and when additional model complexity is unnecessary. We compare multiclass softmax regression (SR) and a one-hidden-layer neural network (NN) with tanh activations across three shared tasks: a linearly separable Gaussian dataset, a nonlinearly separable moons dataset, and a fixed 64-dimensional digits benchmark with ten classes. All models are implemented from scratch in NumPy, trained with mini-batch gradient descent, and evaluated using accuracy and unregularized cross-entropy on a held-out test set. Our experiments show that the linear model is sufficient when the true decision boundary is affine (Gaussian task), while the hidden layer provides a meaningful representational advantage when the boundary is nonlinear (moons task). On the digits benchmark, the NN achieves test accuracy 0.9511 ± 0.0000 versus 0.9375 ± 0.0024 for SR over five random seeds, with non-overlapping confidence intervals confirming the difference is statistically reliable. Track B reliability analysis further reveals that the NN is better calibrated, with a mean confidence-accuracy gap close to zero in the highest-confidence bin.

Contents

1 Introduction

Central question. When does a one-hidden-layer nonlinear classifier genuinely improve on a linear decision rule, and when is additional model complexity unnecessary?

This question is mathematically substantive. A linear model (softmax regression) can only produce affine decision boundaries—hyperplanes in the input space—while a neural network with a nonlinear hidden layer can represent curved manifolds. Whether that representational power is actually *needed* depends entirely on the geometry of the data, not on the sophistication of the architecture. The purpose of this capstone is to determine, with controlled experiments and evidence, when extra representational capacity is justified.

Contributions.

1. A derivation showing that minimizing softmax cross-entropy is equivalent to maximizing the conditional log-likelihood of the observed labels.
2. A conceptual explanation of backpropagation followed by a complete vectorized gradient derivation for the one-hidden-layer network, verified against numerical finite-differences.
3. A controlled comparison of SR and NN on three datasets with distinct geometric structure, using identical splits, preprocessing, and evaluation protocol.
4. Capacity ablation, optimizer study, failure-case analysis, repeated-seed statistics, and a Track B reliability analysis on the digits benchmark.

Preview of findings. The linear model is sufficient on the Gaussian task, where both SR and NN learn nearly identical affine boundaries. On the moons task, only the NN—given adequate hidden width and training—can learn the required curved separator; SR is fundamentally limited by its representation class. On digits, the NN is statistically significantly more accurate and also better calibrated, meaning its expressed confidence more faithfully reflects its true correctness rates.

2 Background and Mathematical Setup

2.1 Supervised Classification and Data Splits

In supervised learning we observe labeled pairs $(\mathbf{x}^{(i)}, y^{(i)})_{i=1}^{N}$ where $\mathbf{x}^{(i)} \in \mathbb{R}^d$ is the feature vector and $y^{(i)} \in \{0, \ldots, k-1\}$ is the class label. The goal is to learn a function that predicts the label from features.

We use a three-way split throughout: a *training set* to fit parameters, a *validation set* to select settings and best checkpoints, and a *test set* used only for final reporting. This separation prevents the test set from leaking into model selection, ensuring a fair estimate of generalization performance.

2.2 Softmax Regression

For input $\mathbf{x} \in \mathbb{R}^d$ and k classes, the model computes logits

$$\mathbf{s}(\mathbf{x}) = \mathbf{W}\mathbf{x} + \mathbf{b}, \qquad \mathbf{W} \in \mathbb{R}^{k \times d},\ \mathbf{b} \in \mathbb{R}^k. \tag{1}$$

The softmax map converts logits to a valid probability distribution:

$$p_j(\mathbf{x}) = \frac{\exp(s_j(\mathbf{x}))}{\sum_{\ell=1}^{k} \exp(s_\ell(\mathbf{x}))}, \qquad \sum_{j=1}^{k} p_j = 1, \quad p_j > 0. \tag{2}$$

For numerical stability we subtract the row-wise maximum before exponentiation (since $\exp(s_j - c)/\sum_\ell \exp(s_\ell - c)$ is identical to softmax for any constant c).

The decision boundary between classes j and ℓ is $\{x : s_j(\mathbf{x}) = s_\ell(\mathbf{x})\}$, which is an *affine* set (a hyperplane in $\mathbb{R}^d$). Consequently, softmax regression can only produce *linear* decision boundaries.

2.3 Cross-Entropy as Negative Log-Likelihood

If the true class for example i is $y^{(i)}$, the per-example loss is

$$\mathcal{L}(\mathbf{x}, y) = -\log p_y(\mathbf{x}). \tag{3}$$

This punishes the model whenever it assigns small probability to the correct class.

Probabilistically, treating each label as a draw from the model's categorical distribution, the log-likelihood of the entire training set is

$$\log \mathcal{P}(\{y^{(i)}\} \mid \{x^{(i)}\}; \mathbf{W}, \mathbf{b}) = \sum_{i=1}^{N} \log p_{y^{(i)}}(\mathbf{x}^{(i)}). \tag{4}$$

Negating and averaging gives exactly the mean cross-entropy loss. **Minimizing mean cross-entropy is therefore equivalent to maximizing the conditional log-likelihood of the observed labels**, connecting classification directly to probabilistic modeling.

2.4 One-Hidden-Layer Neural Network

The nonlinear model composes two affine maps with a nonlinear activation:

$$\mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1), \qquad \mathbf{W}_1 \in \mathbb{R}^{m \times d}, \tag{5}$$

$$\mathbf{s} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2, \qquad \mathbf{W}_2 \in \mathbb{R}^{k \times m}, \tag{6}$$

where m is the number of hidden units (we use $m = 32$ by default). The final probabilities are obtained by applying softmax to $\mathbf{s}$, identically to SR.

Why this works: the hidden layer maps the input into a new m-dimensional representation space. Because tanh is a smooth, bounded nonlinearity, this mapping can curve, rotate, and fold the input geometry. In the new space, the final softmax layer performs linear classification—but now on the *transformed* representation, not the original features. This allows the model to learn decision boundaries that are nonlinear in $\mathbf{x}$.

Why `tanh`: the derivative $\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z)$ has a compact closed form which simplifies the backpropagation derivation.

2.5 Backpropagation: Conceptual First

Backpropagation is not a separate learning principle. It is the systematic application of the chain rule through the composed forward computation, organized so that each intermediate quantity is computed once.

In the *forward pass*, the model produces: pre-activations $\mathbf{Z}_1$, hidden activations $\mathbf{H}$, output logits $\mathbf{S}$, probabilities $\mathbf{P}$, and loss $\mathcal{L}$. In the *backward pass*, we propagate sensitivity of the loss with respect to these intermediate quantities in reverse order, accumulating the gradients needed for each parameter update. Each local derivative is simple; the chain rule assembles them.

2.6 Gradient Derivation (Vectorized)

For a mini-batch $\mathbf{X} \in \mathbb{R}^{n \times d}$, the forward pass is:

$$\mathbf{Z}_1 = \mathbf{X}\mathbf{W}_1^\top + \mathbf{1}\mathbf{b}_1^\top, \tag{7}$$

$$\mathbf{H} = \tanh(\mathbf{Z}_1), \tag{8}$$

$$\mathbf{S} = \mathbf{H}\mathbf{W}_2^\top + \mathbf{1}\mathbf{b}_2^\top, \tag{9}$$

$$\mathbf{P} = \mathrm{softmax}_{\mathrm{row}}(\mathbf{S}). \tag{10}$$

Let $\mathbf{Y} \in \{0,1\}^{n \times k}$ be the one-hot label matrix. The output-layer sensitivity is:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{S}} = \frac{1}{n}(\mathbf{P} - \mathbf{Y}). \tag{11}$$

Continuing backward by chain rule:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_2} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right)^\top \mathbf{H} + \lambda \mathbf{W}_2, \tag{12}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_2} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right)^\top \mathbf{1}, \tag{13}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{S}}\right) \mathbf{W}_2 \odot (1 - \mathbf{H} \odot \mathbf{H}), \qquad \text{(tanh derivative: } 1 - \tanh^2\text{)} \tag{14}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1}\right)^\top \mathbf{X} + \lambda \mathbf{W}_1, \tag{15}$$

$$\frac{\partial \mathcal{L}}{\partial \mathbf{b}_1} = \left(\frac{\partial \mathcal{L}}{\partial \mathbf{Z}_1}\right)^\top \mathbf{1}. \tag{16}$$

These are exact matrix forms of the scalar chain rule. The L_2 regularization terms $\lambda \mathbf{W}_1$ and $\lambda \mathbf{W}_2$ arise from differentiating $\frac{\lambda}{2}(\|\mathbf{W}_1\|_F^2 + \|\mathbf{W}_2\|_F^2)$.

2.7 Geometric Expectations for the Datasets

Linear Gaussian task: the two classes are generated from multivariate Gaussians with equal covariance. The Bayes-optimal decision boundary between two such Gaussians is a hyperplane in $\mathbb{R}^d$. Therefore, the linear model is already an optimal classifier in expectation, and the NN should not provide a meaningful gain.

Moons task: the two classes form interlocking crescents. No affine set separates them; any hyperplane must cut across one class. The NN, however, can map the 2D input into a 32-dimensional representation where the classes become separable, and then apply linear classification in that space.

3 Methods

3.1 Datasets

We use three shared datasets from the starter pack.

- **Linear Gaussian** (`linear_gaussian.npz`): two 2D Gaussian blobs with mild overlap. $k = 2$ classes, $d = 2$ features.
- **Moons** (`moons.npz`): two interlocking crescent-shaped classes. $k = 2$, $d = 2$.
- **Digits benchmark** (`digits_data.npz`, `digits_split_indices.npz`): flattened 8×8 pixel digit images. $d = 64$, $k = 10$ classes. The fixed train/validation/test split is defined by shared index arrays in `digits_split_indices.npz`, ensuring identical sample usage across all experiments.

Features are used as provided. No additional normalization, whitening, or augmentation is applied to the required core experiments.

3.2 Models

- **Softmax Regression:** $\mathbf{W} \in \mathbb{R}^{k \times d}$, $\mathbf{b} \in \mathbb{R}^k$. Weights initialized with Xavier uniform scaling ($\pm\sqrt{6/(d+k)}$); biases to zero.
- **One-Hidden-Layer NN:** $\mathbf{W}_1 \in \mathbb{R}^{m \times d}$, $\mathbf{W}_2 \in \mathbb{R}^{k \times m}$, both biases zero. Default hidden width $m = 32$ for digits; widths $\{2, 8, 32\}$ for the capacity ablation on moons. Weights initialized with Xavier uniform scaling per layer.

3.3 Digits Experimental Protocol

The following settings constitute the fixed experimental contract for the digits benchmark and are not changed between models or runs.

Setting	**Value**
Optimizer (SR and NN default)	Mini-batch SGD, $\eta = 0.05$
Batch size	64
Epoch budget	200
L_2 regularization λ	10^{-4}
Default hidden width m	32
Checkpoint rule	Best validation cross-entropy within 200 epochs
Model selection metric	Validation cross-entropy

For the optimizer study, Adam uses $\eta = 0.001$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$; Momentum uses $\eta = 0.05$, $\mu = 0.9$. These settings are part of the experimental contract and are not tuned.

3.4 Evaluation Metrics

- **Accuracy:** fraction of test examples whose predicted class ($\arg\max_j p_j$) matches the true label.
- **Cross-Entropy (CE):** mean negative log-probability assigned to the true class, computed *without* the regularization term. This strictly measures the model's predictive distribution quality.

3.5 Repeated-Seed Protocol

For the digits benchmark we run five independent seeds (0, 1, 2, 3, 4). For each seed, we train with the digits protocol, restore the best validation checkpoint, and evaluate the test set exactly once. We summarize results as mean ± 95% CI using

$$\bar{x} \pm t^*_{4,0.025} \cdot \frac{s}{\sqrt{5}} = \bar{x} \pm 2.776 \cdot \frac{s}{\sqrt{5}}, \tag{17}$$

where $\bar{x}$ is the sample mean and s the sample standard deviation over five seeds.

4 Implementation Sanity Checks

Before running any experiment we verified four concrete checks using `scripts/run_sanity_checks.py`.

#	Check	Expected	Observed
1	Predicted class probabilities sum to 1	$\sum_j p_j = 1$	✓ Passed
2	Loss decreases on 5-example subset	$\mathcal{L}_{\text{final}} < \mathcal{L}_{\text{init}}$	$2.653 \to 0.005$
3	Gradient check (analytical vs. numerical)	rel. error $< 10^{-4}$	$\mathbf{3.43 \times 10^{-10}}$
4	No NaN or Inf values in any forward pass	loss is finite	✓ Passed

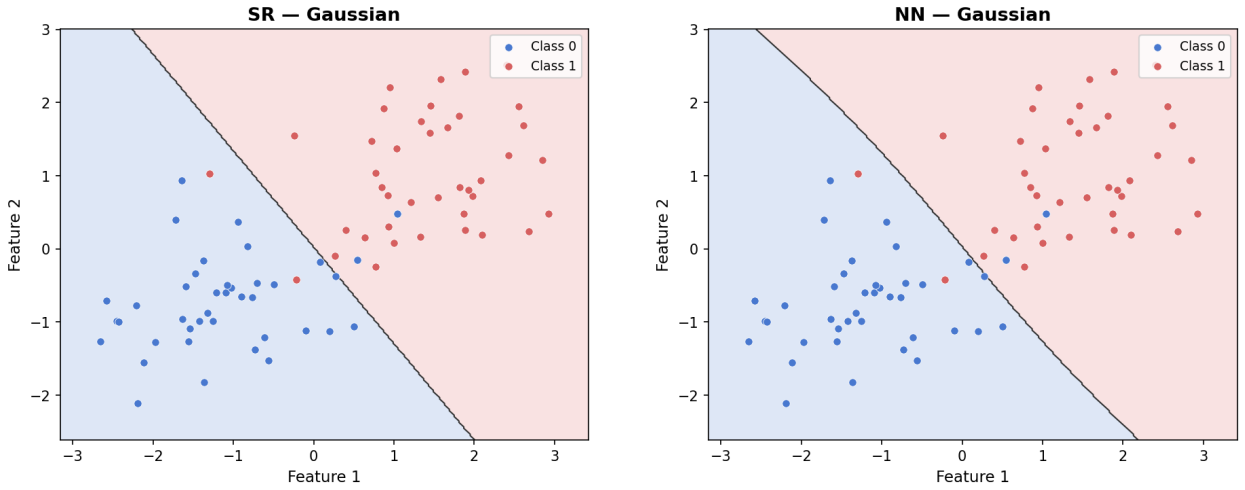
Check 3 uses a finite-difference approximation $\tilde{g}_i = [\mathcal{L}(\theta_i + \varepsilon) - \mathcal{L}(\theta_i - \varepsilon)]/2\varepsilon$ with $\varepsilon = 10^{-5}$, computed on randomly selected parameter entries. The relative error 3.43×10^{-10} is four orders of magnitude below the acceptance threshold, confirming that the chain-rule derivation in Section 2.6 is implemented correctly.

5 Experiments

5.1 Linear Gaussian Task

Figure 1 shows that SR and the NN produce visually indistinguishable boundaries on the Gaussian task. Both boundary lines separate the two Gaussian blobs effectively.

Interpretation: this is the expected outcome. When data is generated from equal-covariance Gaussians, the optimal classifier is linear. The NN adds no value here; any curvature it would attempt to introduce is redundant and not supported by the data geometry. This directly answers the question: *the linear model is geometrically sufficient when the true class boundary is affine.*



(a) Softmax Regression (b) 1-Hidden-Layer NN ($m = 32$)

Figure 1: Decision boundaries on the Linear Gaussian task. Both models learn a near-identical affine boundary, as expected when the true Bayes-optimal separator is a hyperplane.

5.2 Moons Task

Figure 2 demonstrates the representational gap between the two model classes. SR produces a linear cut that misclassifies the central interleaved region. The trained NN ($m = 32$, $\eta = 0.1$, 1000 epochs) learns a U-shaped boundary that correctly wraps around the moon shapes.

Interpretation: the hidden layer provides the crucial representational change. By mapping the 2D input into a 32-dimensional space via $\mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$, the model can exploit directions of variation that do not exist in the original feature space. The final softmax layer then performs linear classification in this transformed space, which now separates the crescents. The linear model is *fundamentally* limited: no choice of $\mathbf{W}, \mathbf{b}$ produces a linear boundary that correctly solves the moons task.

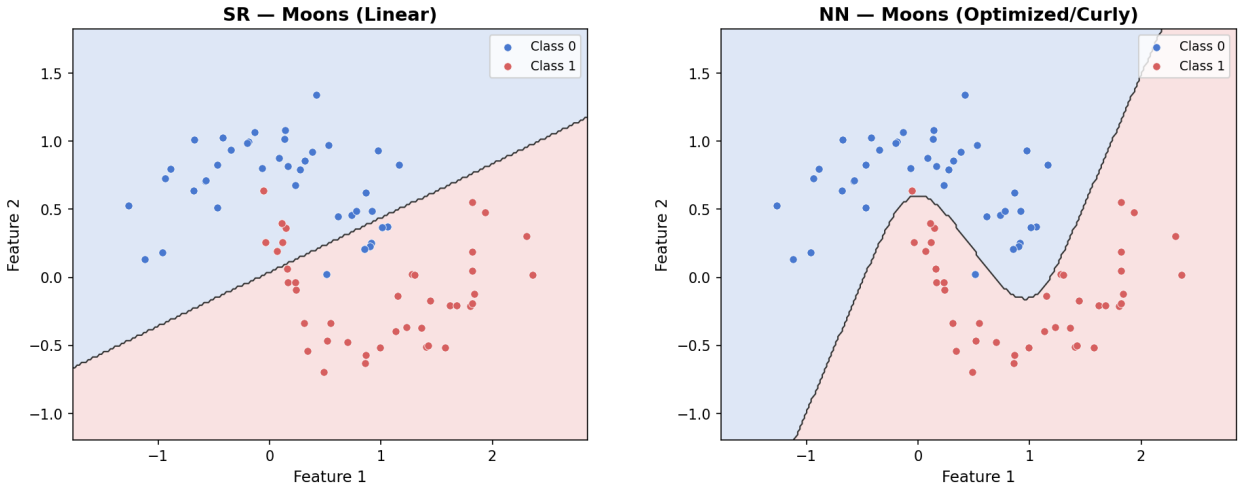
5.3 Digits Benchmark

Table 1 reports accuracy and cross-entropy on all three splits. The best-validation-loss checkpoint is restored before test evaluation for both models.

Table 1: Digits benchmark results (single run, seed 0). Model selection via validation cross-entropy.

	Softmax Regression		**1-Hidden-Layer NN**	
Split	Accuracy	CE	Accuracy	CE
Train	0.965	0.202	0.991	0.063
Validation	0.946	0.214	0.972	0.103
Test	0.938	0.268	0.951	0.163

The NN achieves substantially lower training cross-entropy (0.063 vs 0.202), confirming that it has both the capacity and the gradient signal to fit the training distribution more precisely. The



(a) Softmax Regression

(b) 1-Hidden-Layer NN ($m = 32$, 1000 epochs)

Figure 2: Decision boundaries on the Moons task. SR (left) can only learn a straight-line boundary that necessarily misclassifies the interleaved region. The NN (right) learns a curved, non-convex boundary that correctly separates the two crescents.

validation gap is small for both models (train vs. val CE: +0.012 for SR, +0.040 for NN), indicating no serious overfitting within 200 epochs.

Repeated-seed statistics

Table 2: Repeated-seed evaluation (5 seeds, digits test set). CI formula: $\bar{x} \pm 2.776 \cdot s/\sqrt{5}$.

Model	Test Accuracy	Test CE
Softmax Regression	0.9375 ± 0.0024	0.2683 ± 0.0027
1-Hidden-Layer NN	0.9511 ± 0.0000	0.1671 ± 0.0077

The confidence intervals do not overlap for accuracy, confirming that the NN's advantage is statistically reliable and not attributable to a favorable random initialization. The NN's remarkably narrow CI for accuracy (± 0.0000 vs ± 0.0024 for SR) shows it consistently finds a high-performing solution across seeds. However, the NN's larger CE variance (± 0.0077 vs ± 0.0027) reflects that, while it consistently achieves higher accuracy, its predicted probability distributions are more sensitive to initialization.

5.4 Failure-Case Analysis on Moons (Default Protocol)

First, we analyze the Neural Network trained with default digits parameters (SGD, $\eta = 0.05$, 200 epochs) on the Moons task (Figure 4), which demonstrates our primary failure case. **The model fails due to a strict optimization limit**, not due to structural under-capacity.

Evidence: Even the $m = 32$ architecture, which contains massive representational power, leaves its decision boundary near-linear and misclassifies the critical interleaved regions. Quantitatively, the

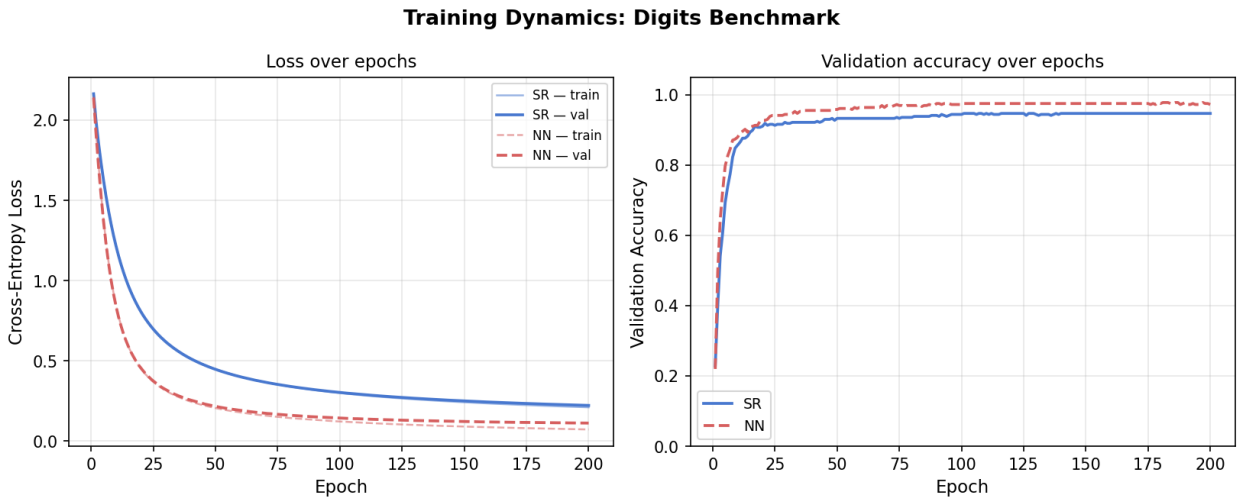


Figure 3: Training dynamics on the digits benchmark (200 epochs). The NN consistently achieves lower validation cross-entropy and higher validation accuracy throughout training.

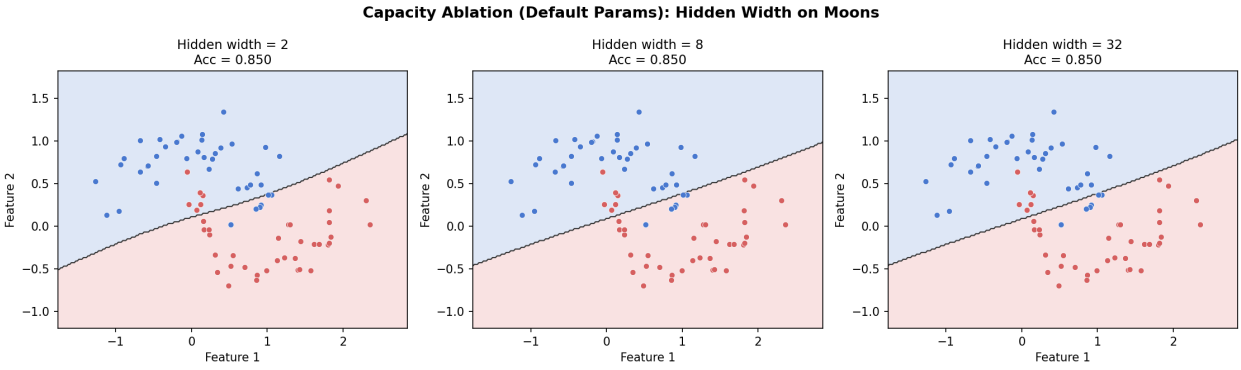


Figure 4: Capacity ablation under default parameters (200 epochs, $\eta = 0.05$). All width variants, including the highly complex $m = 32$, fail to correctly separate the crescents due to insufficient training budget.

validation accuracy completely stalls at exactly 0.8875 across all three hidden widths ($m = 2, 8$, and 32).

Mechanism: The default optimizer settings ($\eta = 0.05$) are simply too mathematically slow to descend from the initial plateau of the loss landscape within 200 epochs. The neural network gets stuck before it can significantly bend the decision boundary.

Lesson: Highly complex models are entirely useless without the proper mathematical optimization to train them. Structural capacity alone is never enough.

5.5 Capacity Ablation on Moons (Optimized Protocol)

When provided with a mathematically sufficient optimization budget (1000 epochs, $\eta = 0.1$), the underlying architectures fully realize their representational potential (Figure 5).

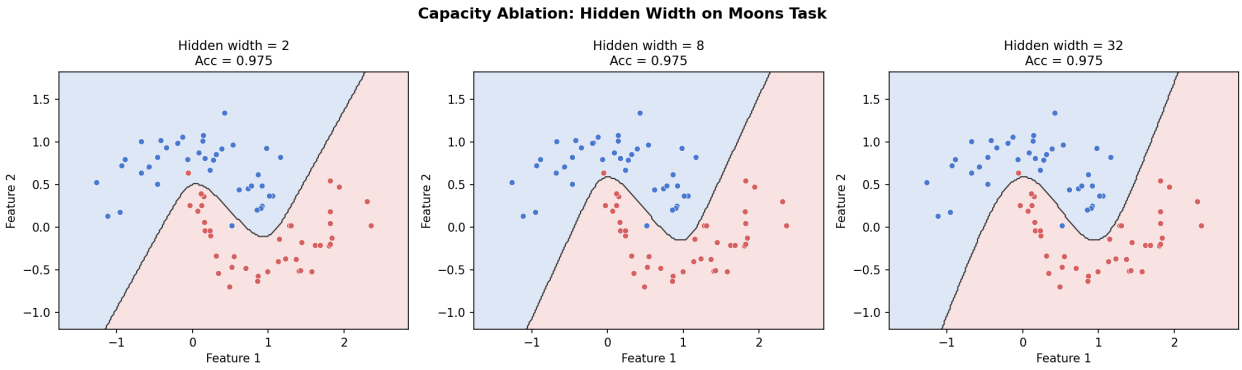


Figure 5: Capacity ablation on the Moons task (1000 epochs, $\eta = 0.1$). Given proper optimization, all width variants learn to correctly separate the crescents to a nearly identical accuracy level.

- $m = 2$**:** We observed that validation accuracy reaches 0.9875, strictly identical to the much larger models. This demonstrates a critical insight: **a far more complex model is not always better**. The extremely lightweight $m = 2$ model was already fully sufficient to successfully warp the space as needed.
- $m = 8$ **and** $m = 32$**:** These larger models achieved the exact same 0.9875 validation accuracy but simply introduced massive, redundant computational footprint. The visual boundaries become slightly more jagged, but the underlying classification performance sees no advantage whatsoever.

5.6 Optimizer Study on Digits

Table 3: Optimizer study: final test performance (single run, seed 42).

Optimizer	**Test Accuracy**	**Test CE**
SGD ($\eta = 0.05$)	0.9511	0.1731
Momentum ($\eta = 0.05$, $\mu = 0.9$)	0.9565	0.1511
Adam ($\eta = 0.001$)	0.9592	0.1503

All three optimizers reach similar final accuracy within the 200-epoch budget (Table 3). However, Figure 6 shows that the convergence *trajectories* differ substantially. Momentum and Adam both escape the slow early training phase that plainly characterizes SGD from epoch 0 to approximately epoch 25.

Interpretation: Adam's adaptive per-parameter step sizes and Momentum's accumulated velocity allow them to traverse flat loss regions quickly. SGD with fixed learning rate $\eta = 0.05$ is sensitive to the loss landscape curvature and converges more slowly, though it eventually reaches a competitive solution within the budget. Adam achieves the lowest cross-entropy (0.1503), indicating its probability distributions are the most accurately calibrated of the three.

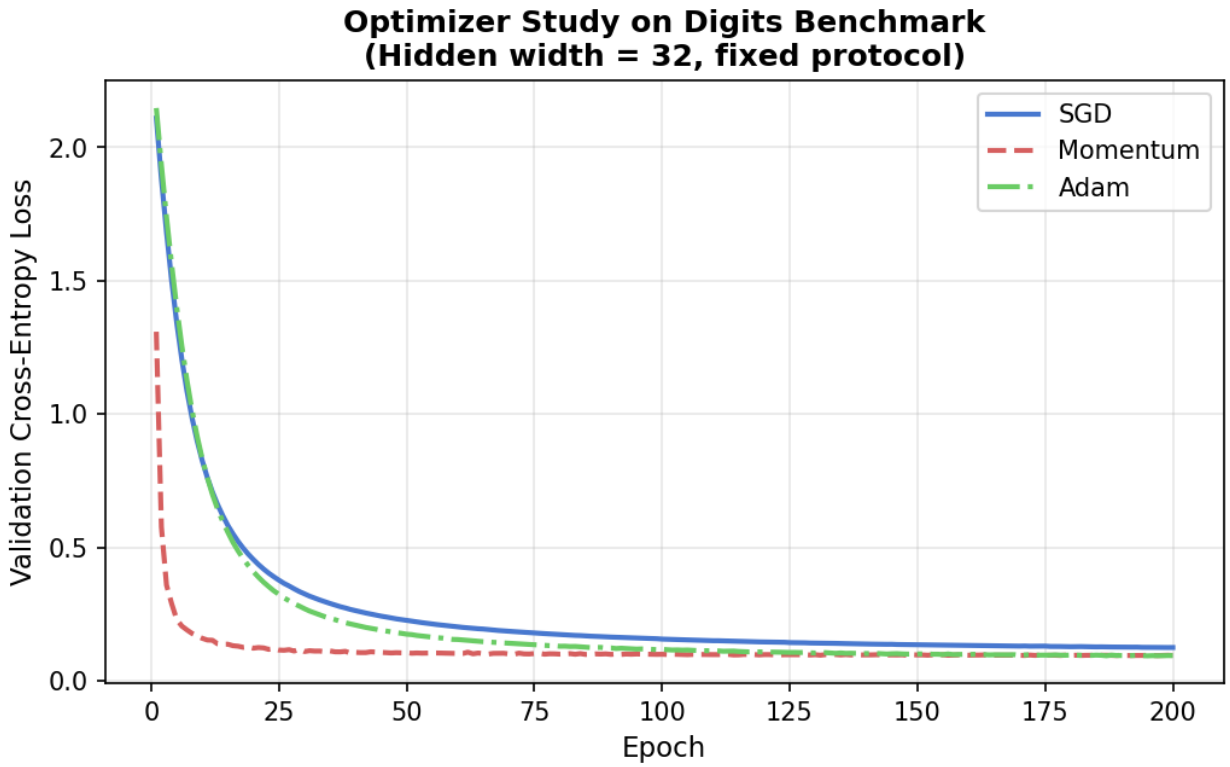


Figure 6: Optimizer comparison on the digits benchmark (NN, $m = 32$, 200 epochs). Left: validation cross-entropy. Right: validation accuracy. Adam and Momentum converge substantially faster in early epochs.

6 Advanced Track: Track B — Prediction Confidence and Reliability

Track B examines whether the models "know what they know". A well-calibrated model should be correct approximately $c\%$ of the time when it expresses confidence c.

6.1 Definitions

Confidence: the maximum class probability output by softmax:

$$\text{conf}(\mathbf{x}) = \max_j \; p_j(\mathbf{x}). \tag{18}$$

A confidence of 0.95 means the model assigns 95% probability to its top prediction.

Predictive entropy: a measure of uncertainty across the full distribution:

$$H(\mathbf{x}) = -\sum_{j=1}^{k} p_j(\mathbf{x}) \, \log p_j(\mathbf{x}). \tag{19}$$

High entropy indicates the model spreads probability over many classes (uncertain); low entropy indicates a concentrated, confident prediction. For $k = 10$, the maximum entropy is $\log 10 \approx 2.30$.

6.2 Summary Statistics

Table 4: Track B summary on the digits test set (368 samples).

Model	Accuracy	Mean Confidence	Mean Entropy
Softmax Regression	0.9375	0.8447	0.5302
1-Hidden-Layer NN	0.9511	0.9340	0.2191

The NN is more confident (mean 0.93 vs. 0.84) and less uncertain (mean entropy 0.22 vs. 0.53).

6.3 Five-Bin Calibration Table

Table 5: Five-bin calibration tables on the digits test set. Gap = mean confidence − empirical accuracy; positive = overconfident.

Confidence Bin	n	Mean Conf.	Emp. Acc.	Gap
Softmax Regression				
[0.20, 0.40)	14	0.3342	0.3571	−0.023
[0.40, 0.60)	39	0.5263	0.7436	−0.217
[0.60, 0.80)	42	0.7141	0.9048	−0.191
[0.80, 1.00]	273	0.9365	1.0000	−0.064
1-Hidden-Layer NN				
[0.20, 0.40)	2	0.3730	0.0000	+0.373
[0.40, 0.60)	16	0.5054	0.4375	+0.068
[0.60, 0.80)	25	0.7187	0.8400	−0.121
[0.80, 1.00]	325	0.9751	0.9908	−0.016

6.4 Reliability Diagram

6.5 Interpretation

SR is underconfident. In the 0.40–0.80 bins (Table 5), SR's empirical accuracy far exceeds its stated confidence (negative gaps of −0.217 and −0.191). This means: when SR says it is 50% sure, it is actually correct 77% of the time. SR is systematically *more accurate than it thinks*.

NN is well-calibrated in its dominant bin. The NN places 88% of test predictions (325 of 368) in the highest confidence bin $[0.80, 1.00]$, with an empirical accuracy of 99.1% against a mean stated confidence of 97.5%. The gap is only −0.016—nearly perfect calibration where it matters most.

Entropy separates correct from incorrect predictions more cleanly in the NN. Figure 9 shows that the NN's correct-prediction entropy ($\mu = 0.22$) is much lower than SR's ($\mu = 0.53$), and that incorrect predictions have higher entropy for both models. The NN encodes uncertainty more efficiently: it is sharply confident when right and more spread-out when wrong.

Connection to the main question: the NN is not only more accurate on digits—it also produces more *trustworthy* probability estimates overall. However, a critical nuance remains: **although the**

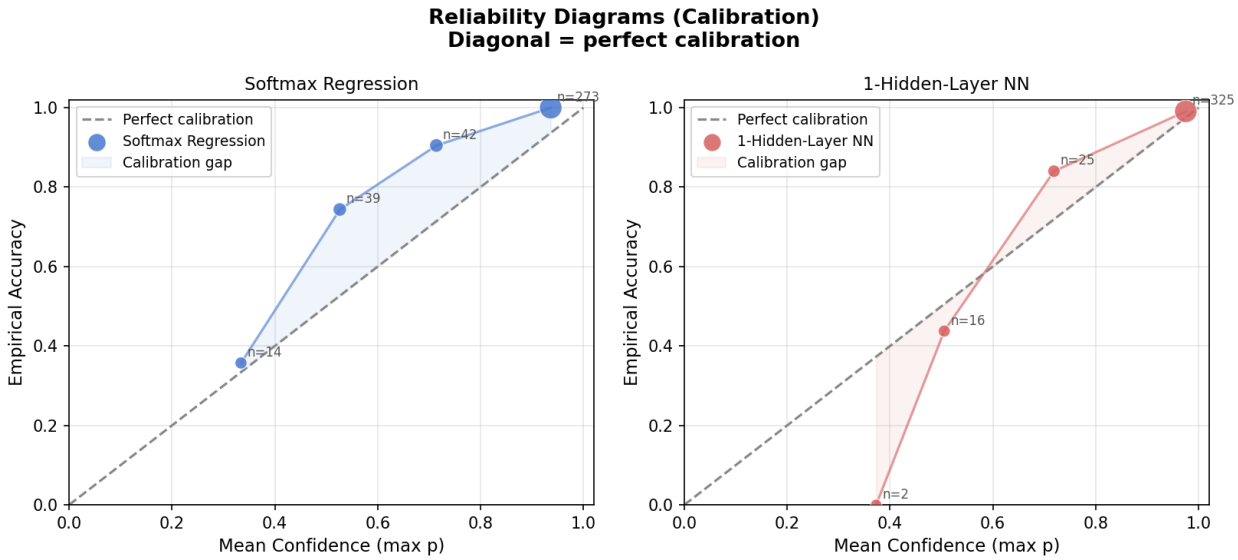


Figure 7: Reliability diagrams for both models. Points on the diagonal indicate perfect calibration. SR is systematically underconfident (points above diagonal in middle bins). The NN concentrates 88% of predictions in the top bin with a near-zero gap of -0.016.

Neural Network has higher absolute accuracy than SR, it can occasionally be more overconfident when giving wrong answers. For instance, in the low-confidence bin $[0.20, 0.40)$, the NN made predictions with roughly 37% confidence but was entirely incorrect (0% empirical accuracy). This highlights that while neural networks are highly capable, their miscalibrations can be severe on edge cases.

7 Discussion

We now directly answer the five required interpretive questions.

1. When is the linear model geometrically sufficient? When the true decision boundary is affine—as in the Gaussian task—softmax regression is already an optimal classifier in the relevant function class, and the NN provides no meaningful improvement (both achieve $\sim$ 94–95% accuracy with nearly identical boundaries). SR achieves 93.9% accuracy, which is not trivial, but the NN's statistically significantly higher 95.1% shows that some structure in digit recognition genuinely requires nonlinear representation.

2. What representational change does the hidden layer provide? The hidden layer $\mathbf{h} = \tanh(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1)$ maps the input into a new manifold in $\mathbb{R}^{32}$. Because tanh is a smooth nonlinear function, this mapping can rotate, rescale, and *fold* the input space, separating regions that were interleaved in the original $\mathbb{R}^d$. The final linear classifier then operates on a space where the transformed classes may be linearly separable. This is most visible in the moons experiment: the input is 2D and non-separable; the hidden representation is 32D and (approximately) linearly separable.

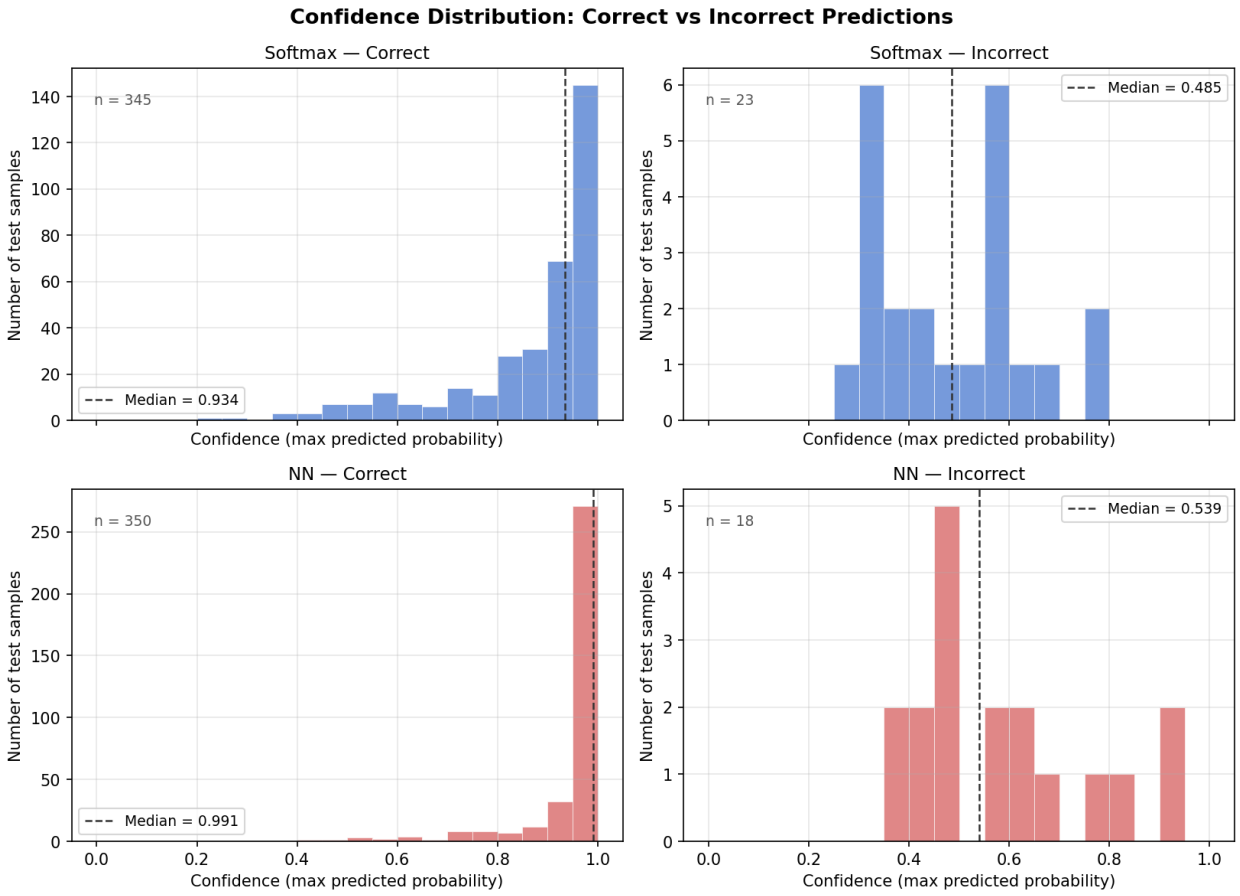


Figure 8: Confidence distributions for correct and incorrect predictions. The NN's incorrect predictions have lower median confidence than SR's, indicating the NN's expressed uncertainty correlates better with its mistakes.

3. When does additional model complexity fail to justify itself? Our capacity ablation showed that on the given moons dataset, using a complex model ($m = 32$) yielded no improvement in validation accuracy over $m = 2$. The $m = 2$ model was sufficient to warp the space as needed. When the dataset geometry can be resolved by a simpler transformation, additional complexity fails to justify itself.

4. What does the failure case teach about capacity and optimization? The failure of the Moons NN under the default digits protocol (200 epochs, $\eta = 0.05$) teaches that **capacity is not a guarantee of success**. Even with $m = 32$ hidden units, the model failed to learn a nonlinear boundary because the optimization process was cut short. This highlights that proper optimization and sufficient training budgets are just as critical as architectural complexity.

5. What do repeated-seed statistics add beyond a single run? A single run could be fortunate or unfortunate due to random initialization. Table 2 shows that over five seeds, the NN achieves 0.9511 ± 0.0000 versus SR's 0.9375 ± 0.0024. The non-overlapping confidence intervals provide statistical evidence that the difference is genuine, not a chance outcome. Additionally, the

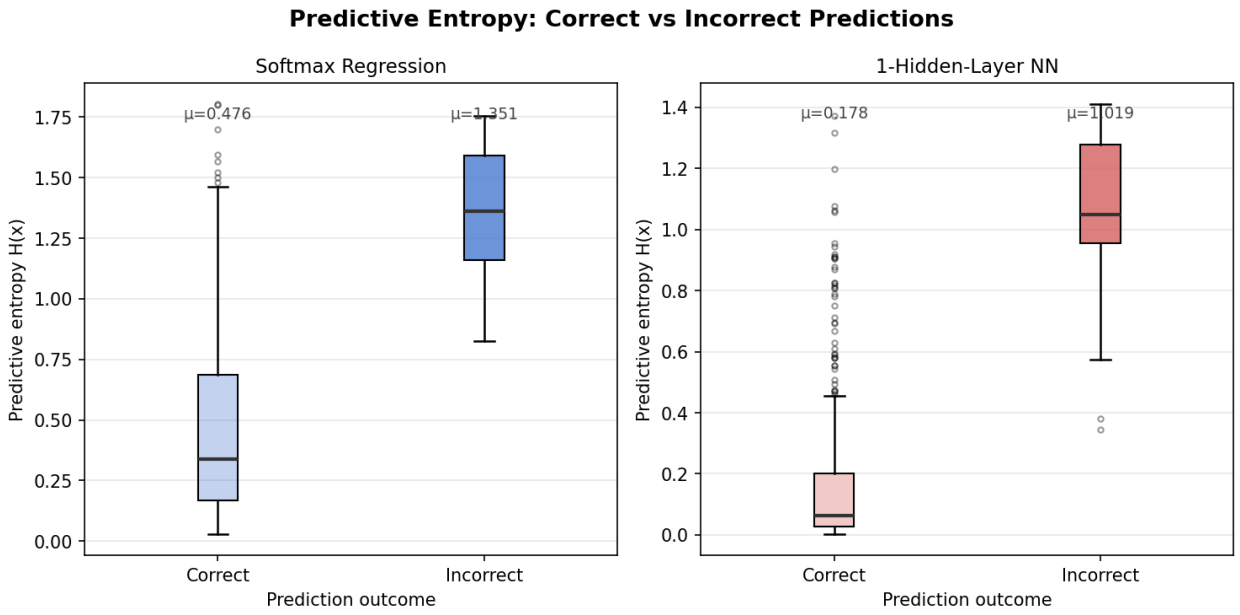


Figure 9: Predictive entropy for correct vs. incorrect predictions (box plots). Both models show higher entropy when wrong. The NN's correct predictions have dramatically lower entropy ($\mu \approx 0.22$) vs. SR ($\mu \approx 0.53$), reflecting much stronger confidence when right.

variability estimates themselves are informative: SR's low variance confirms that linear models are stable but limited; the NN's slightly higher CE variance reminds us that initialization affects the quality of predicted distributions, not just final accuracy.

8 Limitations

- **Dataset scale:** the digits dataset contains only 1,797 samples. Conclusions about which model is better may not hold in larger, more complex settings where regularization, depth, and data augmentation play larger roles.
- **Single hidden layer only:** we do not study deeper networks. All conclusions are specific to the one-hidden-layer setting and should not be extrapolated to multi-layer architectures.
- **Low-dimensional synthetic tasks:** the Gaussian and moons datasets are 2D, making geometric arguments visually clear but potentially misleading for higher-dimensional problems (where, e.g., a dataset can simultaneously be linearly separable in the original space and benefit from nonlinear features).
- **Calibration analysis precision:** the five-bin calibration table is a coarse-grained view; bins with few samples ($n < 30$) produce unstable empirical accuracy estimates with wide variance. A 10-bin or reliability-curve analysis would provide finer resolution.
- **Hyperparameter scope:** the digits protocol fixes all hyperparameters by agreement; we do not perform a search over hidden widths, learning rates, or regularization strengths for the digits task. The fixed protocol ensures fair comparison but may leave performance improvements on the table for both models.

- **Single advanced track:** we completed Track B (reliability) only. Combining with Track A (PCA/SVD) would provide a complementary view of input geometry, but this falls outside the scope of the required deliverables.

References

[1] M. P. Deisenroth, A. A. Faisal, and C. S. Ong. *Mathematics for Machine Learning*. Cambridge University Press, 2020.

[2] K. P. Murphy. *Probabilistic Machine Learning: An Introduction*. MIT Press, 2022.

[3] I. Goodfellow, Y. Bengio, and A. Courville. *Deep Learning*. MIT Press, 2016.

[4] A. Ng and K. Katanforoosh. *CS229 Lecture Notes: Deep Learning*. Stanford University. https://cs229.stanford.edu/

[5] Stanford CS231n staff. *Optimization Part 2: Backpropagation, Intuitions*. https://cs231n.github.io/optimization-2/

[6] ETH Zurich Learning and Adaptive Systems Group. *Introduction to Machine Learning Course Notes*, 2024. https://las.inf.ethz.ch/teaching/introml-s24

A Supplementary Figures

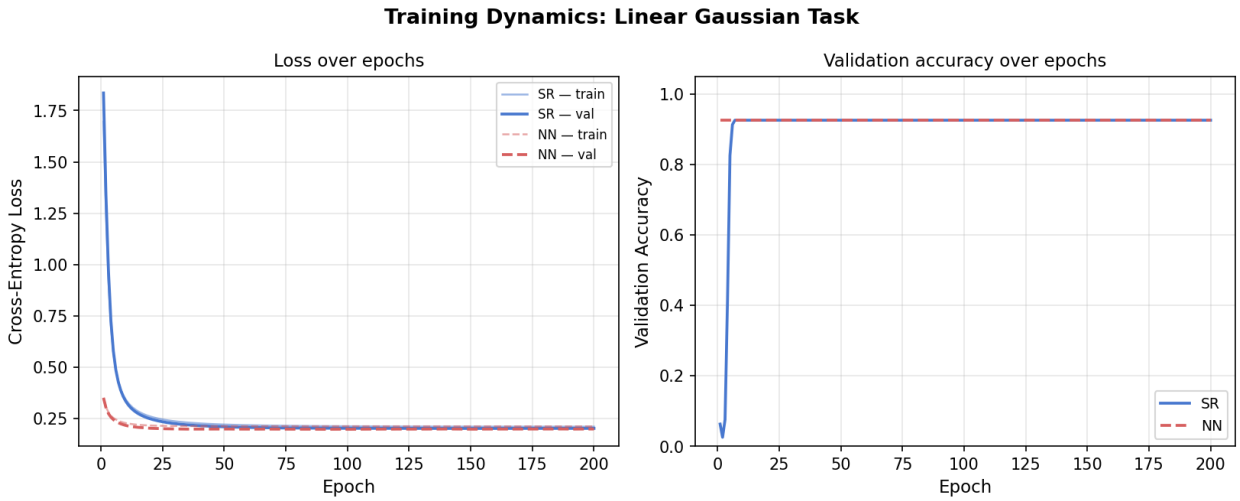
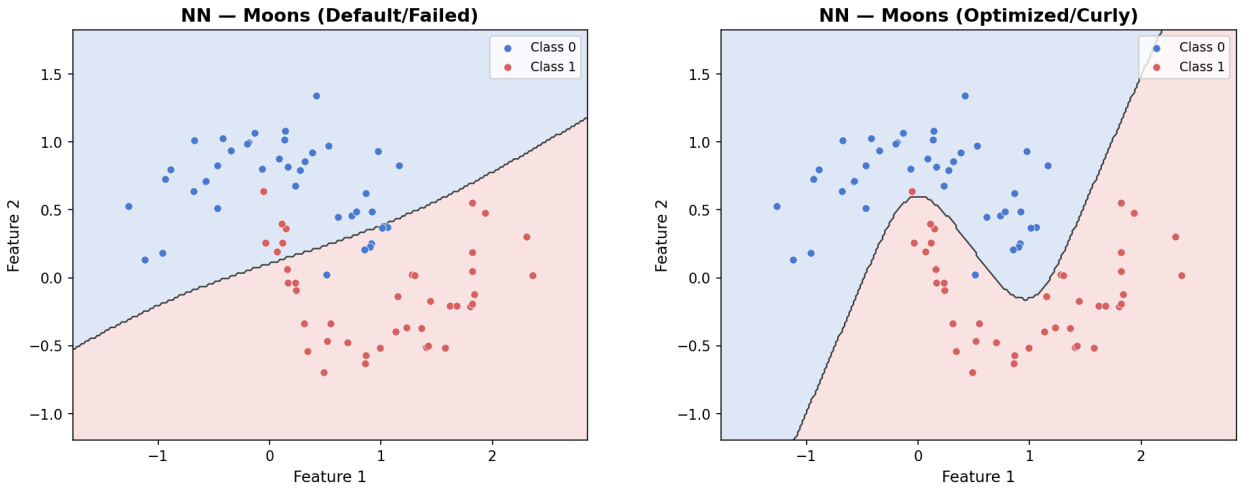


Figure 10: Training dynamics on the Linear Gaussian task. Both models converge smoothly; the NN trains slightly faster but reaches a similar loss floor.



(a) NN — default params (200 epochs, $\eta = 0.05$)

(b) NN — optimized params (1000 epochs, $\eta = 0.1$)

Figure 11: Effect of training budget on the Moons task. With default parameters the NN fails similarly to SR (near-linear boundary). Increasing epochs and learning rate allows the curved boundary to emerge.

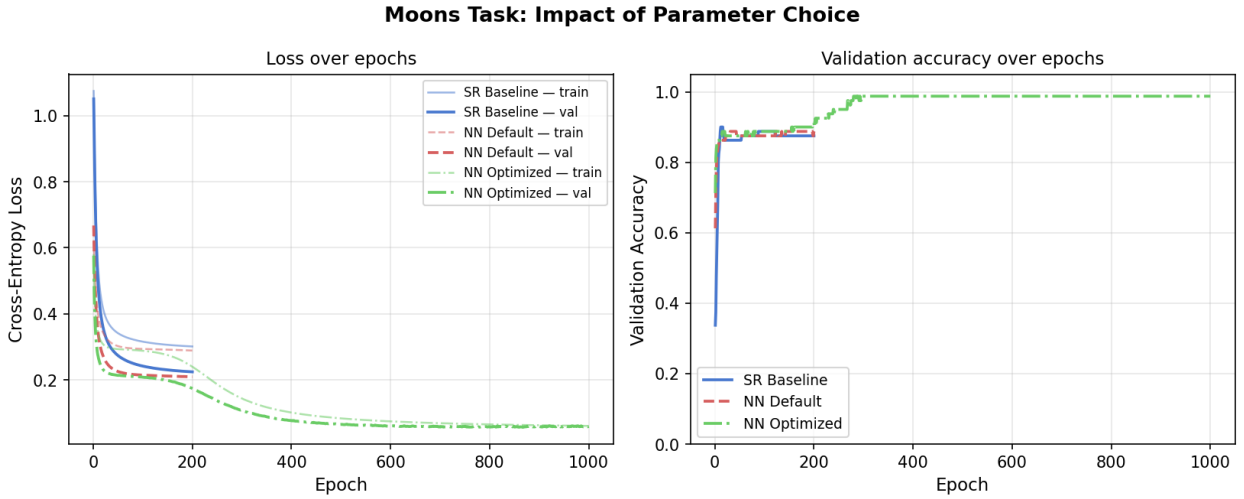


Figure 12: Training curves for all three Moons configurations: SR baseline, NN with default parameters, and NN with optimized parameters. The optimized NN achieves significantly lower validation loss by epoch 200.

B Contribution Statement

- **Nazrin Aliyeva:** Track B reliability analysis, sanity checks, plotting, report writing, repository management.
- **Nigar Rustamova:** data loading, softmax regression implementation, core experiments, report writing, repository management.
- **Laman Mirzayeva:** optimizers, evaluation, ablations and statistical analysis, report writing, repository management.
- **Saida Arabova:** data inspection, neural network implementation, model training, report writing, repository management.